Notes on Trees

Tree Team

February 19, 2024

1 Introduction

A popular model for supervised learning is the Classification And Regression Tree (CART). As in all of machine learning, question regarding how well a model generalizes to unseen data is central. The problem of model generalzation is sometimes called the overfitting problem. One way of evaluating generlization is using cross-validation, which allocates a prespecified fraction of the data for evaluating the trained model. This type of validation has many caveats, for one, there is less data available for training the model. Further, using too little data for validation produces poor estimate of the **generalzation error**. Others have proposed estimating the error using the same data that generated the model using various methods (AIC, BIC etc).

Finally, another approach is to evaluate the generalization by using inference based methods and hypothesis testing. The connection between direct estimation of generalization error and hypothesis test is unclear. However, they aim to do the same thing.

For CART there are a multitude of methods propsed for dealing with overftting using inference. [Shih and Tsai, 2004] use inference to determine the split of a node. Others use traditional scoring methods for choosing splits but use inference to determine the quality of the split, see [Jensen and Schmill, 1997, Hothorn et al., 2006, Neufeld et al., 2022]. However, care should be taken when using inference. [Jensen and Schmill, 1997] argues that mutiple testing across covariates should be accounted for and [Neufeld et al., 2022] warns about *selective inference*. See [Neufeld et al., 2022] for selective inference on CART and [Fithian et al., 2017] for the general theory of selective inference.

Results from the change-point detection literature can be leveraged for constructin trees, since splitting a node is essentially the same as finding a change-point in the data. Specifically, we will use results from [Hawkins, 1977, Yao and Davis, 1986] for inference on trees. There are a multitude of ways for constructing trees using inference, but, below we will focus on a few methods that we argue are reasonable. Furthermore, we will analyse the effect of inference based methods for boosted trees.

2 Background

2.1 Inference

There are in general two broad methods for constructing CART using inference; constructing trees recursively using inference (top-down) and pruning trees using inference (bottomn-up). One can also combine both methods to construct trees.

Constructing trees recursively, can be done by evaluating the quality of a split by some suitable hypothesis test. For instance, let $\mathcal{D} = \{(X_i, y_i)\}_{i=1}^n$ be data available at a particular node of interest. The question is: can $\mathcal{D}$ be partioned into two groups $\mathcal{D}_A, \mathcal{D}_B$ in a statistically significant way. Typically, these groups are chosen by minimizing some loss $l(\mathcal{D}; A, B) = R(\mathcal{D}_A) + R(\mathcal{D}_B)$, for some function R, e.g the L^2 loss $R(\mathcal{D}) = \sum_{i=1}^n (y_i - \bar{y})^2$. Then, a hypothesis test on the estimated partion is performed. The null hypothesis being that there is no difference between the groups. E.g $H_0 : \mu_A = \mu_B$ [Neufeld et al., 2022], both groups have the same mean. Then a suitable test is performed, and the split is accepted if H_0 is rejected with a significance α. This continues recursively, until no more splits can be made.

On the other hand, constructing trees by pruning is done by first building a full tree, for instance by minimizing the loss l described above in a greedy manner. Once a tree is fully constructed, one can for instance, evaluate the quality of splits in the *frotier nodes*(nodes with children that are leaves) by performing a similar hypothesis test as above and removing splits that are insignificant. Finally, the pruning can be stopped once no more leaves can be pruned, this is done by [Jensen and Schmill, 1997].

In both methods of constructing trees, we are in some sense performing a test for each split, this puts us in the multiple testing regime. Here, we can not view each test as being performed independently, in some cases, a test is performed only if a previous test is rejected. However, understanding how these events relate to each other may be too complicated. Instead, we can bound the significance of all test together using *Bonfferoni* corrections. More precisely, we want to bound the *type-1* error in the following way:

$$\begin{aligned}
&\mathbb{P}(\text{rejecting some } H_0^{(i)} | H_0^{(i)} \text{ is true for some } i) = \\
=&\mathbb{P}(\cup_i \{\text{Reject } H_0^{(i)}\} | H_0^{(i)} \text{ is true for some } i) = \\
\leq &\sum_{i=1}^N \mathbb{P}(\{p_i \leq \alpha\} | H_0^{(i)} \text{ is true}) \leq N\alpha \leq \tilde{\alpha}.
\end{aligned}$$

Where, N is the number of nodes in the tree and $\tilde{\alpha}$ measures the total uncertainty. A natural choice for α is then $\alpha = \tilde{\alpha}/N$. However, if the number of nodes N in the tree is not deterministic, this choice of α is not suitable.

Instead, summing the p-values p_i for each node of the tree can be used as α. In the spririt of the top-down and bottomn-up, this means that the

procedure stops just before or after the condition $\sum_{i=1} p_i \leq \tilde{\alpha}$ is satisfied. This is a reasonable stopping rule for constructing trees. A potential issue with this approach is the order in which we consider splits. For instance, building a tree top-down, at each recursion step, there will be M splits to consider, in which order should the p-values for these splits be added to the total? Further, say we accept a split, should the subsequent split be considerd as well? There is no best way to do this. A similar problem arises for pruning. Therefore, a *splitting sequence* needs to be specified.

If we can consider *cost complexty pruning* we can reconcile this problem, since the algorithm specifies a splitting sequence.

[Jensen and Schmill, 1997] proposes the idea of considering multiple testing within a split when there are multiple covariates considered. In fact, a split is determined by comparison to other covariates. For each covariate, a best split is determined, then the chosen split is the best among these individual choices. In this case, a multiple test is being performed. If we consider this, then we can view our problem of constructing a tree as a multiple-multiple test. Should we once again use a Bonfferoni bound $\alpha = \tilde{\alpha}/pN$, where p is the number of covariates and N is the number of nodes. If there are many covariates, will this be too restrictive?

There are many configurations we could consider, however, we will focus on the following:

- Expand this more
- Recursive splitting + summing the smallest p-value.
- ccp-pruning + summing p-values.
- boosting with fixed learner size + summing p-values.
- Adaptive(building with recursion and sum limit) boosting + summing p-values for trees.

Now that we have determined the configurations that should be considered, we can focus on how we should construct the hypothesis test. We have tested creating a test statisic using an estimate of $MSEP$ however we see that this results in some sort of selective inference and since we want to avoid selective inferece all togehter, we have chosen to go ahead with the test proposed by [Hawkins, 1977]. We have not decided yet wheter this test-statistic is free of selective inference but that is the conjecture.

2.2 CART

Below, are some descriptions on various algorithms that may be used in this project.

2.2.1 Cost-Complexity Pruning

Breiman's method of minimal cost-complexity builds a full tree and sequencially prunes nodes to construct a tree that is less complex and that is supposed to generalize better.

If we denote T as a tree and define $R(T)$ as the weighted loss contributions from the leaves of T, we can define a penalized loss $R_\alpha(T) = R(T) + \alpha|T|$, where $|T|$ is the number of leaves of the tree T and α is some hyper-parameter that penalizes complexity. Breiman's algorithm produces the best **subtree** for a given α that minimizes $R_\alpha(T)$. Furthermore, it can be shown that the best trees are nested subtrees T_k of T such that each is optimal for an interval of α. That is, there exits a sequence $\{\alpha_k\}$

$$0 = \alpha_0 < \alpha_1 < \cdots < \alpha_m < \infty \tag{1}$$

such that T_i is the optimal tree w.r.t $R_\alpha(\cdot)$ for $\alpha \in [\alpha_i, \alpha_{i+1})$. Here it should be clear that $T_0 = T$ and T_m is the trivial tree. What this essentially means is that, given some value $\alpha \in [\alpha_i, \alpha_{i+1})$ the tree that minmizes $R_\alpha(\cdot)$ is T_i and T_{i+1} is a subtree of T_i, so starting at $\alpha = 0$ and increasing α will produce a sequence of trees that are pruned subtrees of the preceding until the trivial tree T_m is reached.

The procedure for attaining $\{\alpha_k\}$ and T_k is as follows. Given a non-trivial tree T_i, denote all non-terminal nodes with some t and calculate the gain $g(t, T^{(t)}) = \frac{R(t) - R(T^{(t)})}{|T^{(t)}| - 1}$ for all t, where $R(t)$ is the loss at the node t treated as a trivial tree and $R(T^{(t)})$ is the loss of the tree that is rooted at t. Then $\alpha_i = \min_t g(t, T^{(t)})$. Pruning all nodes where the $g(\cdot) \geq \alpha_i$ produces a new tree T_{i+1}. Now perform the same procedure on T_{i+1} and so on until the trivial tree is reached. This results in a sequence $\{\alpha_k\}$ and a sequnce of nested trees $\{T_k\}$. Two remarks can be made. Firstly, the procedure handles ties in gain by choosing the smallest tree as next in line. Second, the gain is attained from simple algebra by solving for α in the equation $R_\alpha(t) - R_\alpha(T^{(t)}) = 0$.

2.3 Boosting

2.3.1 Adaptive Boosting

When boosting regression trees in the traditional sense, there are two parameters that need to be decided. First, the complexity of the weak learners, e.g. number of leaves of the trees, at each boosting step and second the stopping criterion, that is, step k to stop boosting. The complexity of the tree is choosen heuristically. However, the stopping criterion is typically decided using cross validation. Where the training is stopped as soon as the out-of-sample error starts increasing. Adaptive Boosting Trees (ABT) tackles this problem by proposing an algorithm for boosting with a built-in stopping rule that is based on the complexity of the weak learner. That is, it is sufficient to choose a parameter J

and the procedure will produce boosted trees with at most complexity J and it is conjectured to stop in finite time.

ABT leverages Breiman's pruning method (see 2.2.1) to produce learners of at most J leaves. That is, at each boosting iteration a tree is fitted by choosing the subtree T_k of T such that

$$|T_{k+1}| > J \text{ and } |T_k| \leq J.$$

If $|T_k| = 1$ then the procedure stops. The reason for stopping when this criterion is met, is that the intercept will be identically zero, thus, boosting after this criterion is met will further produce intercepts of zero which is unecessary. Note that in the ABT paper they present a general framework for choosing the response that should be fit at each iteration given a deviance loss. However when an L^2 loss is considered, the reponse at each iteration is the residuals.

A problem raised in the ABT paper is that their procedure sometimes stops early, "stuck in local optima". They propose using a bagging fraction β to overcome this issue.

2.3.2 P-Boosting

Adaptive or non adaptive boosting using inference.

References

[Fithian et al., 2017] Fithian, W., Sun, D., and Taylor, J. (2017). Optimal inference after model selection. *arXiv preprint arXiv:1410.2597*.

[Hawkins, 1977] Hawkins, D. M. (1977). Testing a sequence of observations for a shift in location. *Journal of the American Statistical Association*, 72(357):180–186.

[Hothorn et al., 2006] Hothorn, T., Hornik, K., and Zeileis, A. (2006). Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical statistics*, 15(3):651–674.

[Jensen and Schmill, 1997] Jensen, D. D. and Schmill, M. D. (1997). Adjusting for multiple comparisons in decision tree pruning. In *KDD*, pages 195–198.

[Neufeld et al., 2022] Neufeld, A. C., Gao, L. L., and Witten, D. M. (2022). Tree-values: selective inference for regression trees. *The Journal of Machine Learning Research*, 23(1):13759–13801.

[Shih and Tsai, 2004] Shih, Y.-S. and Tsai, H.-W. (2004). Variable selection bias in regression trees with constant fits. *Computational statistics & data analysis*, 45(3):595–607.

[Yao and Davis, 1986] Yao, Y.-C. and Davis, R. A. (1986). The asymptotic behavior of the likelihood ratio statistic for testing a shift in mean in a sequence of independent normal variates. *Sankhyā: The Indian Journal of Statistics, Series A*, pages 339–353.